

OOP Concepts in the Playwright Test Framework

Analysis of `Playwright_Test` workspace — where classic object-oriented programming concepts are applied in the Page Object Model (POM) structure.

1. Encapsulation

Each page class bundles its locators (data) and actions (methods) together, hiding Playwright selector details from the test files.

- `pages/MakemytripPage.ts` — locators like `fromCityField`, `searchButton` are implementation details; tests only call `searchOneWayFlightBetweenCities(...)`.
- `utils/excel.utils.ts` — `readAndwriteExcel` hides ExcelJS workbook/worksheet handling behind `readExcel()` / `writeExcel()`.
- `pages/BasePage.ts` — `protected readonly page: Page` restricts direct access so only subclasses (not external test code) can touch it directly.

2. Inheritance

`pages/BasePage.ts` is the parent class; every page object extends it to reuse shared behavior instead of duplicating it:

```
export class MakemytripPage extends BasePage { ... }
export class amazonPage extends BasePage { ... }
export class GooglePage extends BasePage { ... }
```

This gives every page the `page` property and the shared `isVisibleSafe()` helper for free.

3. Abstraction

`BasePage` exposes only a minimal, generic contract (`page`, `isVisibleSafe`) and each concrete page class hides *how* a flight search or Excel comparison is actually performed. Test files just call high-level methods like `searchOneWayFlightBetweenCities` or `verifyFlightSearchResultsPageIsLoaded` without knowing the underlying locator/XPath logic.

4. Polymorphism (subtype polymorphism via a common base type)

Because `MakemytripPage`, `amazonPage`, `GooglePage`, `DuckduckgoPage`, and `NaukariPage` all extend `BasePage`, they can be treated uniformly as `BasePage` instances where shared members apply (e.g. `isVisibleSafe`), even though each overrides/extends with its own specific locators and methods. The factory in `pages/app.ts` demonstrates this by uniformly constructing different subclasses the same way (`new XPage(page)`).

5. Composition (object graph via Factory pattern)

`pages/app.ts`'s `createPages()` doesn't use inheritance but **composition** — it builds a single object composed of multiple independent page objects:

```
return {
  google: new GooglePage(page),
  makemytrip: new MakemytripPage(page),
  ...
}
```

This is effectively a Factory function returning a composed set of collaborating objects, a classic OOP creational pattern.

Quick Map

Concept	Where
Encapsulation	Locators + methods co-located in each <code>*Page.ts</code> ; <code>protected</code> page field in <code>BasePage.ts</code>
Inheritance	<code>class XPage extends BasePage</code> in every page class
Abstraction	<code>BasePage.isVisibleSafe()</code> hides try/catch timeout logic from callers
Polymorphism	All page classes share the <code>BasePage</code> type/shared members despite differing implementations
Composition / Factory	<code>createPages()</code> in <code>app.ts</code>